

Lab Activity: Recurrent Neural Networks & LSTMs

Simple RNNs, LSTM Networks, and Language Modelling with NLTK

Estimated Time	40 - 60 minutes
Tools Required	Python 3, Jupyter Notebook, NumPy, NLTK
Key Concepts	RNNs, LSTMs, BPTT, Language Modelling, Tokenisation

Overview

In this lab activity, you will explore the fundamentals of Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. You will work through practical code examples, build your understanding of how these architectures process sequential data, and explore applications in natural language processing using Python and NLTK.

By the end of this activity, you should be able to:

- Explain the architecture and operation of a simple RNN
- Understand back-propagation through time (BPTT) and the vanishing gradient problem
- Describe how LSTM gates (forget, input, output) control information flow
- Use NLTK for text tokenisation, POS tagging, and basic language modelling
- Compare the strengths and limitations of simple RNNs versus LSTMs

Activity Instructions

Task 1: Simple RNNs and LSTM Networks — I Am Trask Tutorial

Visit the **I Am Trask** tutorial linked below. This webpage provides an excellent, visually-rich walkthrough of how to code an LSTM-RNN from scratch in Python. Work through the tutorial and try to replicate the code in your own Jupyter Notebook.

Reference Link:

<https://iamtrask.github.io/2015/11/15/anyone-can-code-lstm/>

Key Focus Areas:

- Study how the simple RNN uses a hidden state to carry information across time-steps
- Understand the role of the sigmoid activation function and weight matrices
- Pay attention to how BPTT computes gradients by unrolling the network through time
- Observe how the LSTM's forget, input, and output gates solve the vanishing gradient problem
- Try modifying hyperparameters (learning rate, hidden layer size) to see how training changes

The key here is to **comprehend the code**, not just run it. Add your own comments to each section explaining what is happening. You may also want to tweak the code to try other things - for example, changing the binary number length, adjusting the learning rate, or modifying the network architecture.

Task 2: RNN Applications in Language Modelling with NLTK

Explore applications of RNNs in language modelling using Python. Work through examples that use **NLTK (Natural Language Toolkit)** and make sure that you understand the workings of NLTK's core functions. Add more comments to expand your understanding.

In your notebook, explore the following NLTK concepts:

- **Tokenisation** - splitting text into words and sentences using `word_tokenize()` and `sent_tokenize()`
- **Part-of-Speech Tagging** - labelling words with their grammatical roles using `pos_tag()`
- **Stopword Removal** - filtering out common words that carry little semantic meaning
- **N-gram Language Models** - building bigram and trigram models to predict the next word in a sequence
- **Text Preprocessing Pipeline** - converting raw text into numerical representations suitable for RNN input
- **Corpus Analysis** - using built-in corpora (Brown, Gutenberg) to analyse word frequency distributions

Task 3: Extend and Experiment

Feel free to amend your notebook with additional exercises. Think about how you could adapt these concepts to your coursework. Note your observations and see if you can extend the tasks to try other algorithms or approaches.

Suggested extensions:

- Increase the binary number length (e.g. 12 or 16 bits) and compare how the simple RNN and LSTM handle longer dependencies

- Implement a GRU (Gated Recurrent Unit) - a simplified alternative to the LSTM with only two gates
- Build a character-level language model that generates text one character at a time
- Extend the bigram model to a trigram model and compare the quality of generated text
- Experiment with different weight initialisation strategies and observe their effect on convergence
- Visualise the gate activations in the LSTM to see how the network learns to manage the carry bit

Recording Your Observations

As you work through the tasks, note your observations. Consider the following questions to guide your reflection:

1. How did the training error change over time for the simple RNN? Was convergence smooth or noisy?
2. What role does the hidden state play? How does the binary addition task demonstrate the need for memory?
3. How does the LSTM compare to the simple RNN in terms of learning speed and stability?
4. What is the vanishing gradient problem, and how do LSTM gates address it?
5. How does a bigram language model compare to what an RNN-based model can achieve? What are the limitations of n-gram approaches?
6. How could you apply RNN or LSTM concepts to your own coursework or projects?

Notes on Feedback & Support

The feedback for these tasks is self-explanatory - you will see your model's error decrease and its predictions improve as training progresses. Should you have any issues while coding, please post them on the **Discussion Board**. The tutor might be able to point you in the right direction of the solution.

Quick Reference: Key Concepts

Concept	Description
Simple RNN	Processes sequences by maintaining a hidden state that is updated at each time-step. Suffers from vanishing gradients on long sequences.

BPTT	Back-Propagation Through Time - the algorithm for computing gradients in RNNs by unrolling the network across all time-steps.
LSTM	Long Short-Term Memory - uses forget, input, and output gates plus a cell state to preserve gradients over long sequences.
GRU	Gated Recurrent Unit - a simplified LSTM with two gates (reset and update). Often performs comparably with fewer parameters.
N-gram Model	A statistical language model that predicts the next word based on the previous n-1 words. Limited by fixed context window.
NLTK	Natural Language Toolkit - a Python library providing tools for tokenisation, tagging, parsing, and access to text corpora.

Primary Resource

Anyone Can Learn To Code an LSTM-RNN in Python:

<https://iamtrask.github.io/2015/11/15/anyone-can-code-lstm/>